

多传感器融合算法教程

面向 Unitree Go2 RSSI 路径进度定位项目

Go2 RSSI / Multi-Sensor Fusion Notes

2026-06-15

目录

阅读方式	3
1 项目中的传感器融合问题	4
1.1 真正要融合的是什么	4
1.2 为什么不是直接估计 x, y	4
1.3 系统总图	4
2 数据格式与时间对齐	6
2.1 RUN_DIR 输出文件	6
2.2 机器人路径进度标签 s	6
2.3 odom 是什么、怎么来的	6
2.4 RSSI 窗口与 odom 的时间对齐	7
2.5 RSSI、IMU、odom 如何同步后进入算法	7
2.6 最直白版：拿到 IMU 和 RSSI 后怎么得到 $\hat{s}$	8
2.7 训练 CSV 的推荐格式	8
3 RSSI 指纹定位：融合算法的测量模型	10
3.1 从 Scheme A 开始：纯 RSSI KNN	10
3.2 加权距离	10
3.3 缺失值不是普通数值	10
3.4 KNN 输出 s	11
3.5 距离度量的选择	11
4 运动约束：从 Scheme A 到 Scheme B	12
4.1 为什么需要运动约束	12
4.2 硬约束：max jump	12
4.3 软约束：continuity penalty	12
4.4 EMA 平滑	12
4.5 confidence：什么时候该相信 RSSI	13
5 IMU 融合：不是积分位置，而是判断运动状态	14
5.1 Go2 IMU 字段	14
5.2 IMU 派生特征	14
5.3 IMU motion gate	14
5.4 RSSI 与 IMU 的相互作用	15
5.5 IMU 控制输入参数起点	16
6 Kalman 与 EKF：把融合写成状态估计	17
6.1 一维路径进度 Kalman Filter	17
6.2 Kalman 预测与更新	17
6.3 EKF：IMU/odom 作为控制输入	18
6.4 参数怎么理解	18

7 更高级的融合路线	19
7.1 粒子滤波：适合多峰 RSSI	19
7.2 HMM/Viterbi：离线轨迹最优	19
7.3 因子图：最终论文级框架	19
7.4 学习型方法放在哪里	20
8 项目落地流程	21
8.1 采集流程	21
8.2 运行 baseline	21
8.3 评估指标	22
8.4 上线前必须做 shadow mode	22
9 常见问题与调参顺序	24
9.1 RSSI-only 误差很大	24
9.2 s 估计跳变	24
9.3 输出太平滑，明显滞后	24
9.4 IMU gate 没效果	25
10 附录：文件与参数对照	26
10.1 关键代码函数	26
10.2 建议阅读的项目资料	26

阅读方式

这份教程的主题是“多传感器融合算法”，但不是泛泛讲 IMU、GPS、雷达融合。它围绕本项目当前 Go2 RSSI 路线复现目标展开：

用 RSSI 估计机器人在一条预定义路线上的全局路径进度 s ，再用 IMU、odom、/go2/path_s 和运动约束让估计更连续、更可信。

本项目不是二维自由定位优先，而是路径跟踪优先。因此本文把状态空间从一开始就定义在路径坐标上。当前核心状态只有 s ，后续扩展只加入横向偏移 e ：

$$\mathbf{x}_t = \begin{bmatrix} s_t \\ e_t \end{bmatrix}$$

其中 s_t 是沿中心线的路径进度，是当前最重要的输出； e_t 是相对中心线的横向偏移，是后续扩展。第一版可以只估计 s_t ，数据采集覆盖中心线左右偏移后再扩展到 (s_t, e_t) 。

本教程和项目代码的对应关系

本文重点对照项目中的 baseline 脚本、Phase A 采集说明、真实 s -only baseline 文档，以及 Go2 RSSI/IMU 采集脚本。尤其是 robot_s_baseline.py，它已经包含 RSSI 指纹、observed mask、距离度量、confidence、IMU motion gate、滑动滤波、Kalman/EKF 等融合雏形。文中凡是写到滤波状态，均以 s 或 (s, e) 为准。

Chapter 1

项目中的传感器融合问题

1.1 真正要融合的是什么

Go2 RSSI 项目当前有这些信号源:

信号	文件/话题	在融合中的角色
RSSI	rss_i_realtime_windows.csv rss_i_raw_stream.csv	主要定位观测。它提供“当前位置像哪个路径参考点”，但噪声大、丢包、多径强。
Go2 IMU	imu_stream.csv, /go2/imu	短时运动状态。适合判断静止/运动、转向、振动、加速度变化，不适合单独长时间积分定位。
Go2 sport state odom	sport_state_straight.csv, /odom	采集阶段的路径标签来源和评估参考。可用于对齐 RSSI 窗口到 s_{go}
路径进度	odom_s_m, /go2/path_s	当前最重要的监督标签和停止条件。它把“机器人走了多远”变成可训练目标。
点云/相机	/go2/pointcloud front_images/	当前主要用于安全和复盘，不作为 RSSI 主定位输入。
Nav2 输出	/cmd_vel_nav costmap / planner log	运动后端/对照信号。Nav2 localization 不应作为 RSSI 主算法输入。

1.2 为什么不是直接估计 x, y

室内 RSSI 多径严重，同一个 x, y 附近的 RSSI 可能随朝向、人体遮挡、机器人姿态变化而变；而路线复现任务天然有一条中心线。用 s 表示“沿路线走到哪里”有三个优点：

1. 标签更容易获得：Go2 运动中可以用 odom_s_m 或 /go2/path_s 对齐 RSSI 窗口。
2. 控制更直接：路线客户端只需要知道当前 s ，再选择 $s + \Delta s$ 的局部目标。
3. 融合更稳定：运动连续性可以自然写成 $s_t \approx s_{t-1} + \Delta s_{to}$

1.3 系统总图

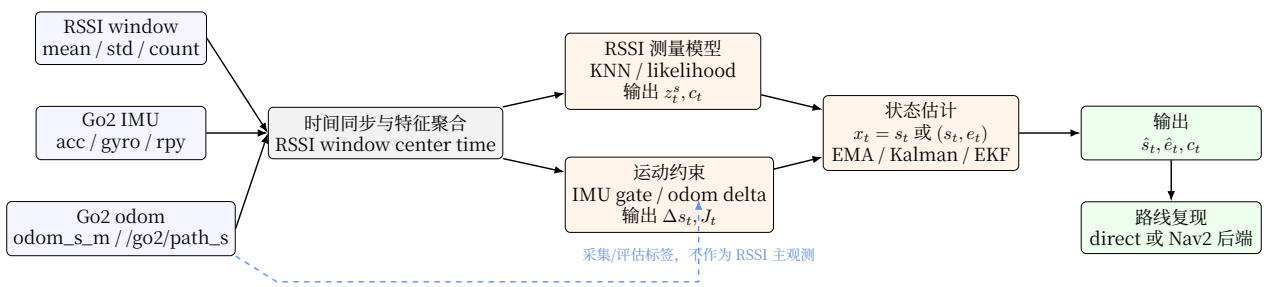


图 1.1: 面向 Go2 路径进度 s 与后续横向偏移 e 的融合流程。RSSI 提供绝对但噪声大的路径观测, IMU/odom 提供时间对齐、短时连续性和运动约束。

Chapter 2

数据格式与时间对齐

2.1 RUN_DIR 输出文件

每次采集会写到一个 RUN_DIR。典型文件如下：

文件	用途
rss_realtime_windows.csv	RSSI 窗口均值、方差、count。融合算法主要输入。
rss_raw_stream.csv	原始 RSSI 包。用于复盘丢包、尖峰和信标可见性。
imu_stream.csv	Go2 lowstate IMU：加速度、角速度、roll/pitch/yaw。
sport_state_straight.csv	Go2 sport state、运动命令、odom_s_m、点云安全状态。采集阶段最重要的 s_{gt} 来源。
run_metadata.txt	本次采集参数：速度、距离、后端、网卡、RSSI 模式等。
front_images/	前置摄像头复盘图像。用于解释异常，不建议第一版加入定位模型。

2.2 机器人路径进度标签 s

在 Phase A 采集中，odom_s_m 是从 Go2 sport state 的 x, y 增量累计得到：

$$s_t = \sum_{i=1}^t \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2}.$$

它和命令积分 cmd_s_m 不一样。cmd_s_m 只是“我命令机器人应该走了多远”，而 odom_s_m 更接近“Go2 状态估计认为实际走了多远”。训练 RSSI 指纹库时，优先使用 odom_s_m 或 ROS 侧 /go2/path_s 作为 s_{gt} 。

2.3 odom 是什么、怎么来的

odom 是 odometry（里程计）的缩写，表示机器人在一个局部连续坐标系中的位姿和速度估计。它不是 GPS，也不是全局真实坐标；它通常会在短时间内连续、平滑，但长时间会漂移。

在本项目中，/odom 由 Go2 bridge 从 Unitree DDS 的 rt/sportmodestate 转换而来。Go2 的 sport state 提供机身位置、速度、姿态等估计量；bridge 把这些字段封装为 ROS 2 标准的 nav_msgs/Odometry，同时发布 odom->base_link TF。这里：

- odom frame 是局部里程计坐标系，起点通常接近本次启动或本次运动的参考位置；

- base_link frame 是机器狗机体坐标系；
- /go2/path_s 或 odom_s_m 是从 odom/spport state 位置增量累计出来的路径长度。

融合算法中，odom 的角色要分清楚：

1. **采集建库时**：odom/path_s 可作为 s_{gt} 标签来源，把 RSSI window 标注到路线进度 s 。
2. **在线融合时**：odom 可提供短时运动增量 Δs^{odom} 或连续性参考，但不应替代 RSSI 成为主定位观测。
3. **评估复盘时**：odom 可和人工标记、视频、Nav2 日志一起作为误差分析依据。

odom 不是绝对真值

如果路线很长、地面打滑、转弯多或 Go2 状态估计漂移明显，odom/path_s 也会偏。论文或报告中应把它称为采集标签或参考标签，而不是绝对 ground truth。必要时可用人工测距、地面标记、视频或外部定位做校验。

2.4 RSSI 窗口与 odom 的时间对齐

RSSI 与 Go2 状态由不同程序记录，不能假设同一行天然对应。正确做法是按时间对齐：

```
rss_i_realtime_windows.csv:
elapsed_sec=12.50, beacon_6_mean=-63

sport_state_straight.csv:
elapsed_sec=12.40, odom_s_m=0.281
elapsed_sec=12.50, odom_s_m=0.284
elapsed_sec=12.60, odom_s_m=0.287

aligned row:
elapsed_sec=12.50, s=0.284, beacon_6_mean=-63
```

如果时间戳有 Unix 时间和 steady elapsed 两套，优先用同一台机器、同一采集脚本内部的 elapsed_sec。跨机器远端采集时，要警惕系统时间不同步；这时可以用脚本启动时间、事件 marker 或运动开始点做校正。

2.5 RSSI、IMU、odom 如何同步后进入算法

实际进入算法的不是“某一瞬间的一条 RSSI”和“某一瞬间的一条 IMU”，而是以 RSSI window 为主时钟组织的一行融合样本。当前算法流程如下：

1. RSSI scanner 按窗口输出一行 rss_i_realtime_windows.csv，包括每个 beacon 的 mean/std/count，以及窗口中心时间 t_k 。若文件只有窗口开始时间，可用 $t_k = t_{start} + 0.5 T_{window}$ 近似。
2. 在 imu_stream.csv 中取 $[t_k - \Delta_{imu}/2, t_k + \Delta_{imu}/2]$ 内的 IMU 样本，计算 $\bar{a}_{dyn}$ 、 $\bar{\omega}$ 、roll/pitch/yaw 波动等派生特征。
3. 在 sport_state_straight.csv 或 /odom 记录中找离 t_k 最近的 odom/path_s，得到采集标签 $s_{gt}(t_k)$ ，也可取前后两点线性插值得到更平滑的标签。
4. RSSI mean/std/count 进入测量模型，输出 RSSI 对路径进度的观测 $z_k^s = \hat{s}_k^{rssi}$ 和置信度 c_k 。
5. IMU 派生特征和 odom 增量进入运动约束，输出“这段时间是否真的在动”、允许最大跳变 J_k 、以及先验增量 Δs_k 。
6. 滤波器更新 s_k 或 (s_k, e_k) ，输出当前路径进度估计。

可以把一行融合样本理解为：

```
time t_k:
RSSI window -> z_s, confidence
IMU window  -> motion_score, static/dynamic flag
odom/path_s  -> label s_gt for training, delta_s prior for online
filter       -> state estimate s_hat, optional e_hat
```

这也是为什么时间戳同步很关键：RSSI window 如果滞后 0.5–1.0 s，而 Go2 以 0.3 m/s 行走，标签就可能错开 0.15–0.30 m；这个误差会直接污染指纹库和在线评估。

2.6 最直白版：拿到 IMU 和 RSSI 后怎么得到 $\hat{s}$

可以把整个融合过程想成四句话。

1. **RSSI 先猜位置。** 当前 RSSI window 会和指纹库里的每个参考点比较。哪个参考点的 RSSI 模式最像当前窗口，算法就先猜当前位置在那个 s 附近，得到 $\hat{s}^{rssi}$ 。同时算法会给一个 confidence：信标看到得多、最近邻距离小、top-K 候选集中，confidence 就高。
2. **IMU/odom 再判断这段时间应该走了多少。** IMU 看加速度和角速度，判断机器狗这 0.5–1 秒是在静止、慢动，还是明显运动。odom 或 /go2/path_s 可以给出这段时间的路径增量。于是算法得到一个运动预测：上一帧在 s_{t-1} ，这段时间大概前进 Δs ，所以先验位置是 $s_{prior} = s_{t-1} + \Delta s$ 。
3. **再决定相信谁多一点。** 如果 RSSI confidence 很高，而且 IMU/odom 也说明机器人确实走了这么远，就让结果更靠近 RSSI 猜测；如果 RSSI 突然跳很远，但 IMU 显示机器人几乎没动，就把结果压回 s_{prior} 附近。
4. **最后输出预测路径进度。** 融合后的结果就是 $\hat{s}_t$ 。它既不是纯 RSSI 的最近邻，也不是纯 odom 积分，而是“RSSI 观测 + IMU/odom 运动预测”共同得到的路径进度。

用最短的伪代码写就是：

```
for each RSSI window:
  s_rssi, confidence = match_rssi_to_fingerprint(rssi_window)
  delta_s = estimate_motion_from_imu_or_odom(imu_window, odom_delta)
  s_prior = previous_s + delta_s
  s_hat = blend(s_prior, s_rssi, confidence, imu_motion_score)
```

所以，RSSI 的作用是“告诉算法像哪里”，IMU/odom 的作用是“告诉算法这段时间应该移动多少、RSSI 的跳变合不合理”，最终输出的是融合后的 $\hat{s}$ 。

2.7 训练 CSV 的推荐格式

REAL_S_BASELINE.md 中定义的真实项目格式是：

```
s,timestamp,beacon_1_mean,beacon_1_std,...,beacon_10_mean,beacon_10_std
```

其中：

- s 是全局路线中心线上的真实路径进度，不是每个文件内部单独从 0 开始的局部进度。
- 缺失 RSSI 原始记录可用 -999，算法内部转成弱信号值，例如 -105。
- beacon_i_mean 是窗口均值；beacon_i_std 是窗口内波动，可用于权重。
- 后续状态扩展只建议加入 e 。若保存 x/y ，它们应作为复盘/可视化字段，不作为本文主状态。

不要把局部 s 当成全局 s

若某个 CSV 文件里的 s 是从该文件起点独立累计出来的局部路径长度，就不能把多个文件直接按 s 排序后当成一条全局路线。Go2 项目必须先定义一条全局中心线，再给每个参考样本统一标注全局 s 。

Chapter 3

RSSI 指纹定位：融合算法的测量模型

3.1 从 Scheme A 开始：纯 RSSI KNN

最简单的定位模型是：

$$z_t^{rssi} \rightarrow \hat{s}_t.$$

其中 z_t^{rssi} 是当前窗口的 RSSI 向量：

$$\mathbf{r}_t = [r_{1,t}, r_{2,t}, \dots, r_{M,t}]^\top.$$

离线建库时，每个参考路径进度 s_j 对应一个指纹：

$$\mathcal{F}_j = \{s_j, \mu_j, \sigma_j, o_j\},$$

其中 μ_j 是各 beacon 的平均 RSSI， σ_j 是标准差， o_j 是 observed rate 或可见性。

3.2 加权距离

robot_s_baseline.py 中的核心距离形式可以写成：

$$d_j(\mathbf{r}_t) = \sqrt{\frac{\sum_{i=1}^M w_{j,i} (r_{i,t} - \mu_{j,i})^2}{\sum_{i=1}^M w_{j,i}}}.$$

权重来自两个直觉：

1. 在某个位置波动小的 beacon 更可信： $1/\max(\sigma_{j,i}, \sigma_{\min})^2$ 。
2. 信号太弱或长期不可见的 beacon 不应该过度影响匹配。

因此代码中使用：

$$w_{j,i} = \text{strength}(\mu_{j,i}) \cdot \frac{1}{\max(\sigma_{j,i}, \sigma_{\min})^2}.$$

3.3 缺失值不是普通数值

RSSI 缺失很常见。把缺失直接当成 -999 参与欧氏距离会摧毁匹配，所以项目里采用：

- 日志层：缺失可记为 -999。
- 算法层：缺失替换成弱 RSSI，例如 -105。
- mask 层：记录 beacon_i_observed，区分“真的弱”和“没收到包”。

- mismatch 层：训练位置可见、测试不可见，或反过来，要给额外惩罚，但不能等同于普通 RSSI 差值。

这对应 robot_s_baseline.py 的 use_mask、missing_mismatch_weight、observed_rate__beacon 等逻辑。

3.4 KNN 输出 s

取距离最小的 K 个状态，用反距离加权：

$$\hat{s}_t^{rsi} = \sum_{j \in \mathcal{N}_K} \alpha_j s_j, \quad \alpha_j = \frac{1/(d_j + \epsilon)}{\sum_{\ell \in \mathcal{N}_K} 1/(d_\ell + \epsilon)}.$$

Scheme A: 纯 RSSI 指纹

```
offline:
  group training rows by global s
  compute beacon mean/std/observed_rate for each s

online:
  read current RSSI window
  compute weighted distance to every fingerprint state
  take top-K nearest states
  output inverse-distance weighted s_pred
```

3.5 距离度量的选择

robot_s_baseline.py 已经支持：

度量	形式	什么时候考虑
Euclidean	平方误差再开方	默认选择，适合 RSSI 差值近似高斯噪声的假设。
Manhattan	绝对误差	对尖峰比欧氏距离更稳健。
Sorensen	归一化绝对差	当不同窗口整体信号强弱漂移明显时可试。
Cosine	方向相似度	更关注 RSSI 模式形状，而不是绝对强度。

第一版实验建议固定 Euclidean 或 Manhattan，不要同时调太多开关。等有稳定 train/test 协议后再系统比较。

Chapter 4

运动约束：从 Scheme A 到 Scheme B

4.1 为什么需要运动约束

RSSI 指纹经常会把两个远处但 RSSI 模式相似的位置混淆。如果每个时刻独立匹配，输出 s 会跳来跳去。真实机器人运动是连续的，所以应该加入约束：

$$s_t \approx s_{t-1} + \Delta s_t.$$

这就是 Scheme B 的核心。

4.2 硬约束：max jump

如果机器人两次 RSSI window 间隔 Δt ，最大速度 $v_{\max}$ ，那么：

$$|s_t - s_{t-1}| \leq v_{\max} \Delta t + \eta.$$

代码里的 `max_jump_s` 就是这个硬约束。它会把候选状态限制在上一帧附近：

```
constrained = abs(candidate_s - prev_s) <= max_jump_s
```

在本项目中，RSSI window 常见参数是 `window_sec=1.0`、`step_sec=0.5`，Go2 直线采集速度常见 0.30m/s。如果每 0.5 s 更新一次，物理上相邻 s 变化大约 0.15 m。实际可给更宽一点，例如 0.5–1.0 m，用来容忍 RSSI 延迟和标签误差。

4.3 软约束：continuity penalty

硬约束可能过于绝对。软约束是在 RSSI 距离上加一项路径连续代价：

$$D_j = d_j^{rssi} + \lambda |s_j - (s_{t-1} + \Delta s_{exp})|.$$

`robot_s_baseline.py` 中对应 `continuity_lambda` 和 `expected_step_s`。

4.4 EMA 平滑

最简单的后处理滤波是指数滑动平均：

$$\hat{s}_t = \alpha \hat{s}_t^{meas} + (1 - \alpha) \hat{s}_{t-1}.$$

`smooth_alpha` 越大，越相信当前 RSSI；越小，越平滑但滞后越明显。

Scheme B: RSSI + 路径连续性 + 平滑

```
for each RSSI window:
    candidates = all fingerprint states
    if prev_s exists:
        keep states within max_jump_s
        add continuity penalty around prev_s + expected_step_s
    pred_s_raw = KNN(candidates)
    pred_s = alpha * pred_s_raw + (1 - alpha) * prev_s
```

4.5 confidence: 什么时候该相信 RSSI

robot_s_baseline.py 的 confidence 由四部分组成:

1. observed beacon 数量是否足够;
2. 最近邻距离是否小;
3. 第一名和第二名距离是否有 margin;
4. top-K 的 s 是否集中。

可以写成:

$$c_t = 0.4c_{obs} + 0.2c_{dist} + 0.2c_{margin} + 0.2c_{spread}.$$

当 confidence 低时, 代码支持把结果往运动先验拉:

$$\hat{s}_t = c_t \hat{s}_t^{rsi} + (1 - c_t)(\hat{s}_{t-1} + \Delta s_{exp}).$$

这就是 confidence_blend。

Chapter 5

IMU 融合：不是积分位置，而是判断运动状态

5.1 Go2 IMU 字段

go2_imu_logger.cpp 输出：

```
timestamp_iso,timestamp_unix,elapsed_sec,  
frame_type,has_acc,has_gyro,has_angle,  
acc_x_g,acc_y_g,acc_z_g,  
gyro_x_dps,gyro_y_dps,gyro_z_dps,  
roll_deg,pitch_deg,yaw_deg
```

IMU 最容易让新手踩坑：加速度二次积分得到位置看起来很诱人，但在真实机器人上漂移会很快爆炸。本项目里更适合先把 IMU 用作“短时运动证据”，不是主位置来源。

5.2 IMU 派生特征

robot_s_baseline.py 中已经实现：

$$a_{norm} = \sqrt{a_x^2 + a_y^2 + a_z^2}, \quad a_{dyn} = |a_{norm} - 1|.$$
$$\omega_{norm} = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}.$$

在 RSSI window 附近取 IMU 均值：

$$\bar{a}_{dyn,t} = \frac{1}{|\mathcal{W}_t|} \sum_{\tau \in \mathcal{W}_t} a_{dyn,\tau}, \quad \bar{\omega}_t = \frac{1}{|\mathcal{W}_t|} \sum_{\tau \in \mathcal{W}_t} \omega_{norm,\tau}.$$

再得到 motion score：

$$m_t = \text{clip} \left(\max \left(\frac{\bar{a}_{dyn,t}}{\theta_a}, \frac{\bar{\omega}_t}{\theta_\omega} \right), 0, 1 \right).$$

当 m_t 接近 0，机器人更可能静止或低运动；当 m_t 接近 1，机器人更可能在明显运动。

5.3 IMU motion gate

代码中 imu_motion_gate 做两件事。它不是让 IMU 直接输出位置，而是让 IMU 决定“RSSI 的这次跳变是否可信”。

第一，根据 motion score 动态改变最大跳变：

$$J_t = J_{static} + m_t(J_{dynamic} - J_{static}).$$

静止时允许的 s 跳变小，运动时允许更大。

第二，根据 motion score 调整 RSSI 和上一帧的融合权重：

$$\alpha_t = \alpha_{static} + m_t(\alpha_{dynamic} - \alpha_{static}),$$

$$\hat{s}_t = \alpha_t \hat{s}_t^{rsi} + (1 - \alpha_t) \hat{s}_{t-1}.$$

这非常适合 Go2 场景：RSSI 有时会突然跳到远处相似指纹，但如果 IMU 认为机器人几乎没动，就应该强烈抑制这种跳变。

5.4 RSSI 与 IMU 的相互作用

RSSI 和 IMU 在融合中承担不同角色：

- RSSI 提供绝对位置感：当前信号模式像路径上哪个 s 或 (s, e) 。
- IMU 提供短时运动证据：这段时间机器人是否移动、是否转身、振动是否异常。
- odom/path_s 提供采集标签和短时先验：训练时标注 s_{gt} ，在线时可作为 Δs 的参考。

因此一次更新可以写成三步。

第一步，RSSI 测量模型给出候选：

$$z_k^s = h_{rsi}(\mathbf{r}_k), \quad c_k \in [0, 1].$$

其中 z_k^s 是 KNN/likelihood 输出的路径进度观测， c_k 是由 beacon 数量、最近邻距离、margin 和 top-K spread 得到的置信度。

第二步，IMU window 给出 motion score：

$$m_k = g(\bar{a}_{dyn,k}, \bar{\omega}_k) \in [0, 1].$$

当 m_k 小，滤波器认为机器人接近静止，RSSI 即使跳到远处也更可能是多径/丢包造成；当 m_k 大，滤波器允许 s 有更大变化。

第三步，把 RSSI 观测和运动先验融合：

$$\Delta s_k = \begin{cases} \Delta s_k^{odom}, & \text{在线有可靠 odom/path_s 参考时,} \\ \Delta s_{exp} m_k, & \text{只有 IMU gate 和期望步长时.} \end{cases}$$

$$s_{k|k-1} = s_{k-1|k-1} + \Delta s_k.$$

RSSI 更新时，IMU 和 confidence 共同决定“拉向 RSSI”还是“保持运动先验”：

$$\hat{s}_k = \beta_k z_k^s + (1 - \beta_k) s_{k|k-1}, \quad \beta_k = c_k(\alpha_{static} + m_k(\alpha_{dynamic} - \alpha_{static})).$$

这条公式表达了两个直觉：RSSI 置信度低时少信 RSSI；IMU 判断接近静止时，也少信 RSSI 的远距离跳变。

RSSI + IMU + odom 同步融合

```
for each RSSI window k centered at t_k:
    # 1. Timestamp alignment
    rssi_features = rssi_window[k]
    imu_features = aggregate_imu(t_k - imu_W/2, t_k + imu_W/2)
    odom_ref = nearest_or_interpolated_odom(t_k)
```



```

# 2. RSSI measurement
z_s, confidence = knn_or_likelihood(rssi_features)

# 3. Motion evidence
motion_score = imu_motion_score(imu_features)
jump_limit = static_jump + motion_score * (dynamic_jump - static_jump)
delta_s_prior = odom_delta_or_expected_step(odom_ref, motion_score)

# 4. State update, state is s or (s, e), not velocity
s_prior = s_prev + delta_s_prior
z_s = clamp_to_local_window(z_s, s_prev, jump_limit)
s_hat = fuse_by_confidence(s_prior, z_s, confidence, motion_score)

```

5.5 IMU 控制输入参数起点

参数	建议起点	含义
imu-window-sec	1.0	每个 RSSI window 附近聚合 1 秒 IMU。
imu-static-acc-threshold	0.04	加速度动态分量低于该值偏静止。需按 Go2 数据调整。
imu-static-gyro-threshold	3.0	角速度模长低于该值偏静止。
imu-static-max-jump-s	0.3--0.5	静止/低运动时允许的最大路径跳变。
imu-dynamic-max-jump-s	1.0--3.0	明显运动时允许的最大路径跳变。
imu-static-alpha	0.10--0.20	静止时当前 RSSI 结果权重。
imu-dynamic-alpha	0.60--0.80	运动时当前 RSSI 结果权重。

Chapter 6

Kalman 与 EKF：把融合写成状态估计

6.1 一维路径进度 Kalman Filter

当只估计路径进度 s 时，可以定义：

$$\mathbf{x}_t = \begin{bmatrix} s_t \end{bmatrix}.$$

状态转移：

$$\mathbf{x}_{t|t-1} = \begin{bmatrix} 1 \end{bmatrix} \mathbf{x}_{t-1|t-1} + \mathbf{w}_t.$$

RSSI KNN 输出作为测量：

$$z_t = \hat{s}_t^{rsi} = \begin{bmatrix} 1 \end{bmatrix} \mathbf{x}_t + n_t.$$

如果 confidence 低，就增大测量噪声：

$$R_t = \frac{R_0}{\max(c_t, 0.05)}.$$

这已经对应 robot_s_baseline.py 的 post_filter=kalman。如果后续扩展到 (s, e) ，只需要把状态改成二维：

$$\mathbf{x}_t = \begin{bmatrix} s_t \\ e_t \end{bmatrix}.$$

此时 e_t 表示横向偏移。

6.2 Kalman 预测与更新

预测：

$$\hat{\mathbf{x}}_{t|t-1} = F \hat{\mathbf{x}}_{t-1|t-1}, \quad P_{t|t-1} = F P_{t-1|t-1} F^\top + Q.$$

更新：

$$\mathbf{y}_t = z_t - H \hat{\mathbf{x}}_{t|t-1},$$

$$S_t = H P_{t|t-1} H^\top + R_t, \quad K_t = P_{t|t-1} H^\top S_t^{-1},$$

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + K_t \mathbf{y}_t.$$

6.3 EKF: IMU/odom 作为控制输入

robot_s_baseline.py 的 post_filter=ekf 版本更贴近本项目。它先从 IMU/motion score 和 odom 得到一个 Δs_t^{imu} 或 Δs_t^{odom} ，再预测：

$$u_t = \Delta s_t^{imu/odom}, \quad s_{t|t-1} = s_{t-1} + u_t,$$

然后仍用 RSSI KNN 输出的 $z_t = \hat{s}_t^{rssi}$ 做更新。严格说，当前一维版本更接近带运动先验的 Kalman；称 EKF 是为了给后续加入非线性路线几何、局部曲率或 (s, e) 耦合预留接口。

当前在线融合主循环

```
for each RSSI window at time t:
    rssi_measurement = KNN(fingerprint_map, rssi_vector)
    confidence = score(observed_dims, distance, margin, topK_spread)
    imu_motion_score = aggregate_imu(t - W/2, t + W/2)
    control_input_u = delta_s_from_imu_or_odom(imu_motion_score, odom_delta)

    predict:
        s_prior = s_prev + control_input_u
        P_prior = F P_prev F^T + Q

    update:
        R = R0 / max(confidence, 0.05)
        s_post = KalmanUpdate(s_prior, rssi_measurement, R)

    publish:
        /path_s_estimate = s_post
```

6.4 参数怎么理解

参数	代码默认值	调参直觉
kalman-process-var	0.05	越大越相信运动模型会变化，输出更灵活；越小越平滑。
kalman-measurement-var	4.0	RSSI 测量基础噪声。越大越不相信 RSSI，越小越跟随 KNN。
confidence-blend	off	开启后低置信度 RSSI 会更靠近运动先验。建议在 RSSI 尖峰明显时测试。
post-filter	none	可选 sliding_mean、sliding_median、kalman、ekf。

Chapter 7

更高级的融合路线

7.1 粒子滤波：适合多峰 RSSI

RSSI 指纹经常多峰：两个不同位置可能 RSSI 很像。Kalman Filter 假设状态近似单峰高斯，而粒子滤波可以保留多个候选。

粒子表示：

$$\{s_t^{(n)}, w_t^{(n)}\}_{n=1}^N.$$

预测：

$$s_t^{(n)} = s_{t-1}^{(n)} + \Delta s_t + \epsilon_t^{(n)}.$$

观测权重：

$$w_t^{(n)} \propto \exp\left(-\frac{d^{rssi}(s_t^{(n)})^2}{2\sigma_r^2}\right).$$

优点是能处理“当前位置可能在 12 m 或 48 m”的模糊；缺点是参数和计算复杂度更高。建议在 Scheme B 和 Kalman 稳定后再做。

7.2 HMM/Viterbi：离线轨迹最优

如果先做离线评估，可以把每个 s bin 当成一个离散状态，RSSI 距离当观测代价，运动约束当转移代价。Viterbi 求整条轨迹最优：

$$\min_{s_{1:T}} \sum_t C_{obs}(s_t, z_t) + \sum_t C_{trans}(s_t, s_{t-1}).$$

这非常适合分析数据集上“理论上融合能做到多好”，但在线控制时要改成递推版本。

7.3 因子图：最终论文级框架

当系统同时有 RSSI、IMU、odom、路径中心线、可能的 e 偏移时，因子图会更清晰。这里的优化状态仍限定为 $\mathbf{x}_t = [s_t, e_t]^\top$ ：

$$\min_{\mathbf{x}_{1:T}} \sum_t \|r_t - h_{rssi}(\mathbf{x}_t)\|_{\Sigma_r}^2 + \sum_t \|\mathbf{x}_t - f(\mathbf{x}_{t-1}, u_t)\|_{\Sigma_u}^2 + \sum_t \|z_t^{odom} - h_{odom}(\mathbf{x}_t)\|_{\Sigma_o}^2.$$

第一版不需要直接上因子图，但可以把它作为论文叙事的终点：RSSI 是全局但噪声大的观测，IMU/odom 是局部连续约束，路线几何是结构先验。

7.4 学习型方法放在哪里

后续可以用神经网络替换 KNN 的测量模型：

$$\hat{s}^{rssi}, c = f_{\theta}(\mathbf{r}).$$

但建议保留同样的融合外壳：网络只输出 RSSI 测量和不确定性，IMU/odom 仍通过滤波器或约束层融合。这样更容易解释，也更容易和 Scheme A/B 公平比较。

Chapter 8

项目落地流程

8.1 采集流程

推荐逐步采集：

1. 定义一条中心线，确定全局 $s = 0$ 到终点。
2. 固定 beacon 位置；移动 beacon 后必须重新建库。
3. 先用 direct 后端做 1 m、3 m、10 m 验证。
4. 正式路线至少正向采 3-5 次。
5. 用 elapsed_sec 把 RSSI window 对齐到 odom_s_m。
6. 按 s 分桶或按真实参考点聚合，生成训练 CSV。
7. 每条测试轨迹单独评估，不要把测试轨迹混成一条伪路径。

8.2 运行 baseline

Scheme A:

```
python3 robot_s_baseline.py \  
  --train your_train.csv \  
  --test your_test.csv \  
  --k 3 \  
  --max-jump-s 0 \  
  --smooth-alpha 0 \  
  --out baseline_outputs/robot_predictions_A.csv \  
  --map-out baseline_outputs/robot_map_A.csv
```

Scheme B:

```
python3 robot_s_baseline.py \  
  --train your_train.csv \  
  --test your_test.csv \  
  --k 3 \  
  --max-jump-s 1.0 \  
  --smooth-alpha 0.4 \  
  --continuity-lambda 0.2 \  
  --expected-step-s 0.15 \  
  --out baseline_outputs/robot_predictions_B.csv \  
  --map-out baseline_outputs/robot_map_B.csv
```

IMU/odom gate + EKF(s):

```
python3 robot_s_baseline.py \
  --train your_train.csv \
  --test your_test.csv \
  --imu imu_stream.csv \
  --imu-motion-gate \
  --post-filter ekf \
  --confidence-blend \
  --max-jump-s 1.0 \
  --expected-step-s 0.15 \
  --out baseline_outputs/robot_predictions_imu_ekf_s.csv \
  --map-out baseline_outputs/robot_map_imu_ekf_s.csv
```

8.3 评估指标

主指标:

$$err_t = |\hat{s}_t - s_t^{gt}|.$$

汇总指标:

- mean error;
- median error;
- p75 / p90 error;
- jump count: $|\hat{s}_t - \hat{s}_{t-1}|$ 超过阈值的次数;
- runtime per update;
- confidence 与 error 的相关性。

论文或报告中第一张对比表建议:

方法	mean	median	p75	p90
RSSI KNN				
RSSI KNN + max jump				
RSSI + continuity + EMA				
RSSI + IMU gate				
RSSI + IMU/odom EKF(s)				

8.4 上线前必须做 shadow mode

在线闭环前, 先让系统只输出 $\hat{s}$, 但不控制机器狗:

```
RSSI window -> fusion node -> /path_s_estimate
human/video/odom -> compare estimate but do not command Go2
```

确认:

- $\hat{s}$ 基本单调或符合运动方向;
- 静止时不会大跳;
- 丢包时 confidence 下降;

- p90 error 在可接受范围内;
- 估计异常时运动后端能限速或停止。

Chapter 9

常见问题与调参顺序

9.1 RSSI-only 误差很大

优先检查：

1. beacon 是否太少或几何分布太差；
2. 训练和测试时 beacon 是否移动；
3. 人是否挡在机器人和 beacon 中间；
4. 缺失值是否正确从 -999 转成 missing_rssi；
5. 是否把多个测试轨迹错误混成一条连续路径；
6. s 标签是否是全局路径进度。

9.2 s 估计跳变

按顺序加：

1. max-jump- s ；
2. continuity-lambda；
3. smooth-alpha；
4. confidence-blend；
5. imu-motion-gate；
6. post-filter=kalman 或 post-filter=ekf。

9.3 输出太平滑，明显滞后

可能原因：

- smooth-alpha 太小；
- kalman-measurement-var 太大；
- max-jump- s 太小；
- IMU motion score 偏低，导致一直按静止处理；
- RSSI window 太长，测量本身延迟。

9.4 IMU gate 没效果

看 *_predictions.csv 中这些列:

- imu_acc_dynamic_mean
- imu_gyro_norm_mean
- imu_motion_score
- imu_is_static
- effective_max_jump_s

如果 motion score 长期接近 0 或 1, 说明阈值不合适。先画出静止段和运动段的 acc_dynamic、gyro_norm 分布, 再定阈值。

Chapter 10

附录：文件与参数对照

10.1 关键代码函数

函数	作用
load_csvs()	读取训练/测试 CSV，处理 -999 缺失，生成 observed mask。
build_fingerprint_map()	按全局 s 聚合 beacon mean/std/observed rate，生成指纹库。
make_weights()	根据 RSSI 强度和 std 计算 beacon 权重。
compute_distances()	支持 Euclidean、Manhattan、Sorensen、Cosine 距离。
compute_confidence()	根据可见 beacon、距离、margin、top-K spread 计算置信度。
attach_imu_features()	把 IMU window 聚合到每个 RSSI row，生成 motion score。
predict_s()	主融合循环：KNN、连续性、confidence、IMU gate、后处理滤波。
summarize_errors()	输出 mean/median/p75/p90。

10.2 建议阅读的项目资料

- REAL_S_BASELINE.md
- PROJECT_PLAN_AB.md
- RSSI_BASELINE.md
- GO2_PHASE_A_STRAIGHT_LINE.md
- GO2_NAV2_INTEGRATION.md
- robot_s_baseline.py
- rssi_fingerprint_baseline.py
- scripts/go2_collect_rssi_imu.sh
- go2_sdk/go2_imu_logger.cpp